Embedded Linux & System Software Interview Handbook

Chapter 6 - Device Tree & Platform Drivers (Part 5)

Topics: Device Tree Debugging, Platform Driver Troubleshooting and Board Bring-up

6.8 Debugging the Device Tree

During board bring-up, one of the first tasks is to verify that the bootloader has loaded the correct Device Tree Blob (DTB). An incorrect DTB can prevent drivers from probing or hardware from functioning correctly.

Useful Debugging Tools

Tool / File	Purpose
dtc	Compile and decompile Device Trees
fdtdump	Inspect DTB contents
/proc/device-tree	View live Device Tree
/sys/firmware/devicetree/base	Kernel Device Tree view
dmesg	Kernel boot and driver logs
/proc/iomem	Verify MMIO regions
/proc/interrupts	Inspect interrupt registration

Typical Platform Driver Debugging Workflow

1. Verify the correct DTB was loaded.
2. Confirm the device node exists.
3. Check the compatible string.
4. Verify probe() execution using dmesg.
5. Validate MMIO resources.
6. Confirm clocks, GPIOs and interrupts are configured.
7. Test register access using readl()/writel().

Common probe() Failures

• Incorrect compatible string.
• Device node marked as status="disabled".
• Missing clocks or reset controllers.
• Invalid interrupt specification.
• Incorrect MMIO address.
• Driver not compiled into the kernel or module not loaded.

Driver Binding

Linux automatically binds a platform driver when the compatible string in the Device Tree matches an entry in the driver's of_match_table. Developers can inspect driver bindings through sysfs and kernel logs.

Real Board Bring-up Checklist

✓ Bootloader loads correct DTB.
✓ Serial console available.
✓ Memory map verified.
✓ GPIO configuration validated.
✓ Clocks enabled.
✓ Interrupts received.
✓ Driver successfully probes.
✓ Device node accessible from user space.

Senior Interview Scenarios

Scenario 1: probe() is never called. How would you debug it?
Scenario 2: Driver probes successfully but hardware does not respond.
Scenario 3: readl() always returns 0xFFFFFFFF. What could be wrong?
Scenario 4: Interrupt never arrives although request_irq() succeeds.

Key Takeaways

Successful BSP development requires a systematic debugging approach. Most board bring-up issues can be traced to incorrect Device Tree entries, resource initialization failures, or platform driver configuration problems. Understanding these workflows is highly valuable in senior Embedded Linux interviews.